

Multi-Protocol Traffic Generation and Analysis: A Containerized System for HTTP/2, QUIC, MQTT and Raw TCP/UDP

Amine Ahamri
Mobile & Internet Computing
Hochschule Rhein-Waal

Abstract—This report describes a containerized system for generating and controlling network traffic. It covers five protocols at once: HTTP/2, QUIC, MQTT, TCP and UDP. 15 Docker containers make up the system. A central controller reads YAML traffic profiles and drives four generators through a REST API. The system supports four sending patterns: constant, periodic burst, linear ramp, and Poisson-distributed random intervals. It also runs an autonomous Adaptive Control loop that reacts to error rates and latency, plus a stealth mode for the TCP/UDP generator that hides its statistical fingerprint. This report covers the architecture, the implementation decisions behind it, the configuration format, and the Wireshark-based traffic analysis.

Index Terms—HTTP/2, QUIC, MQTT, Docker, traffic generation, Wireshark, behavioral fingerprinting, adaptive control

I. INTRODUCTION

A normal office network never carries just one protocol. At any moment there is web traffic, IoT sensor data, video calls and background sync jobs all sharing the same wire. Studying how these protocols behave together, how they fail, and how easy they are to tell apart from the outside, needs a testbed that can actually produce that mix on demand and let you turn the dials while it runs.

This project builds exactly that testbed, based on the assignment in [1]. It generates traffic for five protocols at the same time: HTTP/2 [2], QUIC/HTTP-3 [3], [4], MQTT [5] with all three QoS levels, and raw TCP and UDP. Everything runs as 15 Docker containers orchestrated with Docker Compose [6], started with one `docker-compose up`. A central controller loads a YAML profile and tells four traffic generators what to send and how fast; a web dashboard lets you change any of that live, without restarting anything.

Beyond the basic rate and payload-size knobs, the system implements four sending patterns (constant, periodic burst, linear ramp and Poisson-distributed random intervals), a stealth mode that makes the raw TCP/UDP generator statistically indistinguishable from normal traffic, and an autonomous Adaptive Control loop that scales a generator's rate up or down based on the error rate it is currently seeing, with no human in the loop.

The rest of this report is organized as follows. Section II describes the architecture. Section III walks through the implementation decisions that mattered, and why they were made that way. Section IV documents the configuration format. Sections V through VII present the Wireshark-based traffic analysis, the failure-visibility results and the single- versus multi-machine comparison. Section VIII reflects on what the data actually showed about protocol behavior, and Section IX documents how AI tools were used while building this.

II. SYSTEM ARCHITECTURE

The system has 15 containers on one Docker bridge network called `mic-net`. Containers find each other by service name through Docker's internal DNS, so `gen-http2` can simply call `http://target-http2:8080` without anyone configuring an IP address anywhere.

Ten of those containers carry the core pipeline described below; the remaining five are the Mosquitto broker and four `tshark`-based analyzer sidecars (one per target's network namespace), introduced at the end of this section. The ten core containers fall into four groups. The **controller** (`FastAPI`, port 8000) is the brain: it reads the YAML profile, drives the four generators through their REST APIs, and runs the Adaptive Control loop. This loop is a real feedback mechanism, not just a timer working through YAML phases. Every few seconds the controller checks each generator's current error rate against a threshold and, if it is crossed, scales that generator's rate down by a bounded multiplier on its own, no human involved. Section III-E gives the full description. The four **generators** (`gen-http2`, `gen-quic`, `gen-mqtt`, `gen-tcpudp`) each keep their own state and expose the same four endpoints: `start`, `stop`, `patch the config`, `get the status`. They are built on `httpx` (HTTP/2), `aioquic` [7] (QUIC), `paho-mqtt` [8] (MQTT) and plain Python sockets (raw TCP/UDP). The three **target services** (`target-http2`, `target-quic`, `target-tcpudp`) receive traffic alongside the separately-counted Mosquitto broker [9]. `target-http2` runs on `Hypercorn` [10], the only ASGI server in the project that supports

HTTP/2 natively. The **metrics collector** (Flask, port 9090) aggregates everything into one `/metrics` response, and the **dashboard** (a single static HTML file served over port 3000) polls the controller every two seconds and renders the live state.

Reporting to the metrics collector is not symmetric across these containers, and it is worth being precise about who reports and why. All four generators post their own send-side counters (packets sent, error count, current rate) to the collector, since the controller's Adaptive Control loop reads those numbers. `target-http2` and `target-quic` also post what they received, because both run custom code that can be instrumented freely. Mosquitto cannot do this: it runs the stock `eclipse-mosquitto:2` image with no custom instrumentation hook, so it stays a black box. Every MQTT number in this report, throughput, latency, QoS-level counts, comes only from `gen-mqtt` on the sending side, never from the broker itself. `target-tcpudp` stays equally silent, but for a different reason: it is meant to disappear cleanly when stopped, and the resulting TCP RST is exactly what Section VI needs to see in a Wireshark capture. A reporting connection on top of that would just be one more thing that breaks when the container goes down, so it was left out on purpose.

Fig. 1 shows how a request actually travels through the system. The dashboard never talks to a generator directly; every change goes through the controller, which is what makes it possible to log every configuration change with a timestamp, as the assignment requires.

The same decoupling that makes this diagram work on one machine is what makes the two-machine deployment in Section VII possible without changing a single line of code. Every generator already reaches its target through an environment variable (`TARGET_URL`, `TARGET_HOST`, or `BROKER_HOST`), never through a hardcoded address. Splitting the system across two hosts is therefore just a matter of pointing those variables at a real IP instead of a Docker service name, as described in Section VII; no overlay network or Swarm setup is needed for two machines on the same LAN.

The remaining five containers sit outside the request-handling pipeline shown in Fig. 1: Mosquitto, already introduced above as the target `gen-mqtt` and `target-tcpudp`'s silent counterpart, and four `tshark`-based **analyzer** sidecars (`analyzer-http2`, `analyzer-quic`, `analyzer-mqtt`, `analyzer-tcpudp`), one per target's network namespace (`network_mode: "service:target-X"`, so each sidecar sees exactly the traffic its target sees, nothing more). Each runs a live capture filtered to its own protocol's port, classifies packets purely by port number, and POSTs a running total plus a one-second-bucketed history to the Metrics Collector every second through the same `/analysis/update` endpoint the generators already use for self-reported counters. This gives every number in Section V two independent sources to cross-check against each other: what the generators say they sent, and what `tshark` actually observed on the wire.

III. IMPLEMENTATION DECISIONS AND TRADE-OFFS

A. Connection reuse vs. a fresh connection per packet

The HTTP/2 generator opens one `httpx` client with HTTP/2 enabled and keeps it open for the whole run, firing several requests at once over that single connection to actually use multiplexing rather than just claiming to. The QUIC generator does the same thing for the same reason: a QUIC handshake costs at least one extra round trip, and if the generator reconnected for every request that handshake cost would swamp the latency numbers and make QUIC look artificially slow next to HTTP/2 and MQTT, both of which also keep a connection open.

The TCP/UDP generator does the opposite on purpose. For every single TCP packet it opens a new connection, sends, and closes it again. This was not an oversight. A fresh connect/close cycle produces a real SYN, SYN-ACK and FIN sequence in the capture, which is exactly what is needed later for the Failure Visibility analysis and for telling raw TCP apart from the application-level protocols in a protocol hierarchy view.

B. Mode and pattern as two separate knobs

The TCP/UDP generator splits "what does a packet look like" from "how is it paced over time". `mode` controls size and base interval: normal mode uses a fixed 512 bytes every 100 ms, stealth mode draws a random size between 64 and 1400 bytes with Poisson-distributed timing. `pattern` independently controls the sending cadence on top of that: constant, periodic burst, an extra random gap, or a linear ramp. Keeping these two orthogonal made it possible to test, for example, stealth-mode sizing combined with a burst cadence, which would not have been possible if mode and pattern were the same setting. The other three generators (HTTP/2, QUIC, MQTT) do not have a separate mode the way TCP/UDP does, but they share the same `pattern` field and the same four options, so periodic bursts and ramps are available on every protocol, not just the one that originally needed them for behavioral fingerprinting.

C. The Poisson pattern on every protocol, not just one

The assignment only asks for a random, Poisson-distributed pattern somewhere in the system. This implementation puts it on all four generators using `random.expovariate(1/interval)` (or NumPy's exponential distribution for the TCP/UDP generator), which produces exponentially distributed gaps between sends with the same average rate as the constant pattern. The reason to extend it everywhere was to be able to show one single Wireshark capture where HTTP/2, QUIC and MQTT all use Poisson timing at once, alongside TCP/UDP stealth mode, rather than only ever demonstrating it on one protocol.

D. Warmup/cooldown and the per-protocol ramp pattern are two separate mechanisms

An early version of this system tried to satisfy the assignment's "ramping (linear increase over time)" sending pattern

TABLE I
PER-PROTOCOL CONFIGURATION FIELDS

Protocol	Fields
http2	rate, payload_size, method_get_pct, concurrent_streams, get_paths, post_paths, pattern, burst_rate, burst_duration, burst_interval, ramp_start_rate, ramp_end_rate, ramp_duration, fault_rate, extra_latency_ms
quic	rate, payload_size, stream_count, use_Ortt, pattern, burst_rate, burst_duration, burst_interval, ramp_start_rate, ramp_end_rate, ramp_duration, fault_rate, extra_latency_ms
mqtt	rate, payload_size, qos, topic_count, qos_distribution, pattern, burst_rate, burst_duration, burst_interval, ramp_start_rate, ramp_end_rate, ramp_duration, fault_rate, extra_latency_ms
tcp/udp	tcp_rate, udp_rate, packet_size, mode, mean_interval, min_size, max_size, tcp_ratio, pattern, burst_size, burst_interval, ramp_start_rate, ramp_end_rate, ramp_duration, fault_rate, extra_latency_ms

a linear ramp on all four protocols, then adaptive control), `http2_heavy.yaml` (HTTP/2 under load, then an HTTP/2 + QUIC ramp phase, then a burst phase), and `mqtt_heavy.yaml` (a QoS 0 vs. QoS 2 vs. mixed comparison, then an MQTT burst phase, then an MQTT ramp phase).

`pattern` is the same four-way choice on every protocol: constant, periodic burst, random (Poisson), or ramp. The first three were already implemented on at least one generator from the start; periodic burst and ramp were extended to all four so that any protocol can demonstrate any sending pattern, not only the ones that happened to need it first. Ramp is distinct from the global warmup/cooldown period described above: warmup/cooldown ramps every generator’s rate together, once, at the very edges of a run, whereas `pattern: ramp` ramps one generator’s rate between `ramp_start_rate` and `ramp_end_rate` for the duration of whichever phase sets it, independently of every other protocol active in that phase.

Most field names in the YAML match the generator’s Pydantic model exactly and are forwarded unchanged. Two exceptions exist for readability: `method_distribution`: {GET: 70, POST: 30} on the `http2` block, and `topics`: [...] on the `mqtt` block. Generators only understand `method_get_pct` and `topic_count` respectively, not a named distribution or a list of topic names, so the controller’s `_phase_to_gen_configs()` translates both before forwarding the phase config: the GET/POST split is normalized into a single `method_get_pct` percentage, and the topic list’s length becomes `topic_count` (the topic *names* themselves are not configurable – the generator only rotates through a fixed built-in set – so the YAML list is used purely as a readable way to say how many topics should be active). Any other key that does not match the model is still silently ignored rather than rejected, since the models accept extra keys for forward compatibility but the generators only apply the ones they recognize.

Configuration is not locked to the YAML once the system is running. `POST /config/load` switches the active profile

and applies phase one immediately if the system is already started. `PATCH /generator/{name}` forwards any subset of the fields in Table I to one generator. The dashboard wraps every one of these fields in a slider, toggle or text input that calls the same two endpoints, debounced so that dragging a slider does not flood the controller with requests.

V. TRAFFIC ANALYSIS RESULTS

This section presents the five required Wireshark captures. Each subsection below has the exact setup already filled in, since that part does not depend on the actual capture, only on what to insert once the capture exists.

A. Protocol Distribution

Capture at least 60 seconds with the balanced profile running all four generators. Export Statistics → Protocol Hierarchy as a screenshot.

[INSERT] Screenshot of the Protocol Hierarchy and the percentage breakdown of TCP vs. UDP, and within TCP the HTTP/2 (port 8080), MQTT (port 1883) and raw TCP shares, plus the UDP-only QUIC share (port 4433).

B. Temporal Analysis

Capture 120 seconds with the `http2_heavy` profile, which includes a periodic-burst phase (burst rate 400 for 5 seconds every 30 seconds). Export Statistics → I/O Graph with one line per protocol filter.

[INSERT] I/O graph screenshot with the burst peaks marked, plus a second capture of a warmup period and a Random/Poisson phase for comparison against the constant and burst shapes.

C. Behavioral Fingerprinting

Capture two 30-second windows of the TCP/UDP generator alone: one in `mode=normal`, one in `mode=stealth`. Optionally extend this with the `balanced_with_stealth` phase, which runs stealth mode together with the random pattern on HTTP/2, QUIC and MQTT at the same time.

[INSERT] Packet-length histograms and inter-arrival-time distributions for both modes, with an explicit statement of which characteristics are distinguishable (e.g. the fixed 100ms spacing in normal mode) and which overlap with real client traffic.

D. Failure Visibility

Capture across a `docker stop target-http2` event during normal operation. Filter on `tcp.flags.reset==1` for the TCP RST, or look for repeated SYN to port 1883 if stopping `mosquitto` instead.

[INSERT] Measured time from the stop command to the first visible failure indicator, and which protocol-specific signal appeared. See Section VI for the deeper discussion of this capture together with the Adaptive Control reaction.

E. Multi-System Deployment

Capture on the physical network interface of either lab machine (not `docker0/br-*`) while `docker-compose.targets.yml` runs on Machine B and `docker-compose.generators.yml` runs on Machine A with `TARGET_B_IP` pointing at Machine B.

[INSERT] Capture screenshot plus measured latency per protocol, compared against an equivalent single-machine run. See Section VII for the full comparison.

VI. FAILURE ANALYSIS RESULTS

[INSERT] Consolidated discussion of the Failure Visibility capture from Section V-D. Cover: the per-protocol failure signal observed (TCP RST, MQTT reconnect attempts, QUIC connection-close frames), how long it took to become visible, and what the unaffected protocols looked like during the outage. If Adaptive Control was enabled during the same run, add the controller's logged decisions (error rate, multiplier, action) and how quickly the affected generator's rate was scaled down and back up once the target recovered.

VII. SINGLE-MACHINE VS. MULTI-MACHINE COMPARISON

[INSERT] Quantitative table: latency_ms per protocol on a single machine (loopback) vs. on two machines (real Ethernet hop). Add a short note on what only becomes visible in the multi-machine capture: real host IP addresses instead of Docker's 172.x.x.x range, no shared Docker bridge, and genuine Ethernet framing under load.

VIII. REFLECTION ON PROTOCOL BEHAVIOR UNDER DIFFERENT TRAFFIC PATTERNS

This section should pull together what Sections V through VII actually showed, not repeat them. Questions worth answering once the data is there:

- Which protocol was easiest to fingerprint in normal, non-stealth conditions, and why?
- Did the random pattern change the I/O graph shape equally for every protocol, or did some (HTTP/2 firing several streams per cycle, for instance) keep some residual periodicity?
- Did the measured packet-to-message ratio for MQTT match the expected 1x for QoS 0 and 4x for QoS 2?
- How did Adaptive Control's reaction time compare to the raw, protocol-level failure detection time (TCP RST in under a second vs. the configured check interval)?
- What would change about distinguishability if TLS were used everywhere, including MQTT and raw TCP?

[INSERT] Two to four paragraphs answering the points above with concrete numbers from your own captures.

IX. AI USAGE DOCUMENTATION

I used Anthropic's Claude throughout this project, working directly in the repository through an agentic coding session with file and shell access.

For the code, I used it to scaffold the first version of the `docker-compose.yml` layout, the controller's FastAPI skeleton and the four generators' REST contract (start, stop, patch config, get status), and then iterated on that by hand and through more prompts. The Adaptive Control loop, the warmup/cooldown ramp, the random pattern on all four generators and the per-protocol fault injection were built the same way, one function at a time, with a Python syntax check after every change. The OpenAPI/Swagger documentation was generated from the Pydantic models and FastAPI decorators and kept in sync with a small regeneration script run after every API change.

For the documentation, I had it review the README and the docs against the actual code and fix what no longer matched, for example an old claim that the TCP/UDP generator used Scapy when the real implementation just uses plain sockets.

This report itself was drafted the same way: the architecture, implementation and configuration sections came from the verified source code, while the traffic analysis, failure analysis and multi-machine sections were left as placeholders on purpose, since those need real Wireshark captures that only I can produce on lab hardware.

I reviewed every change before accepting it and ran the syntax checks myself. I also went through the entire codebase module by module in a separate walkthrough session to make sure I could explain every part of it on my own, not just recognize that it works.

[INSERT] Replace this paragraph with your own honest account before submission, including anything I missed above.

REFERENCES

- [1] Hochschule Rhein-Waal, "Project: Multi-protocol traffic generation and analysis," Mobile & Internet Computing course assignment, Summer Term 2026.
- [2] M. Belshe, R. Peon, and M. Thomson, "Hypertext transfer protocol version 2 (http/2)," RFC 7540, IETF, 2015.
- [3] J. Iyengar and M. Thomson, "Quic: A udp-based multiplexed and secure transport," RFC 9000, IETF, 2021.
- [4] M. Bishop, "Http/3," RFC 9114, IETF, 2022.
- [5] OASIS, "Mqtt version 5.0," OASIS Standard, 2019.
- [6] Docker Inc., "Docker compose documentation," <https://docs.docker.com/compose/>.
- [7] aioquic contributors, "aioquic: A quic and http/3 implementation in python," <https://github.com/aioquic/aioquic>.
- [8] Eclipse Foundation, "paho-mqtt: Eclipse paho mqtt python client," <https://github.com/eclipse/paho.mqtt.python>.
- [9] —, "Eclipse mosquitto: An open source mqtt broker," <https://mosquitto.org/>.
- [10] P. Jones, "Hypercorn: An asgi/wsgi server," <https://github.com/pgjones/hypercorn>.